

SkyTrace — Autonomous Drone Delivery ATC System

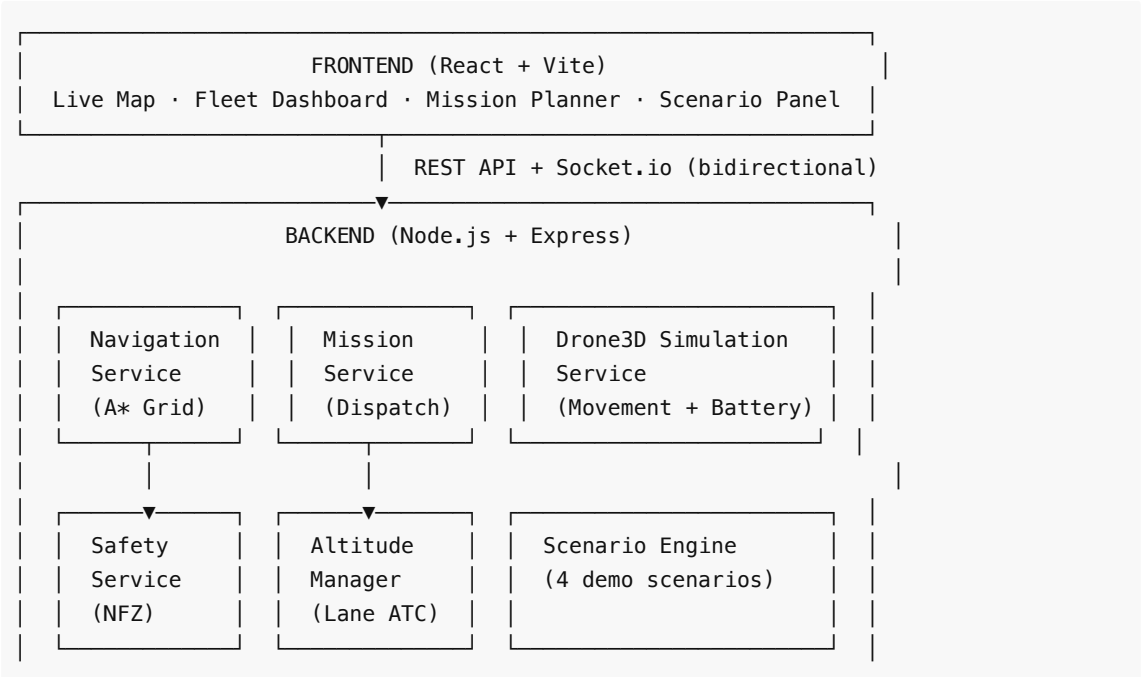
IIT Kanpur | Team: Kratika · Gaurav · Abhishek · Mahee

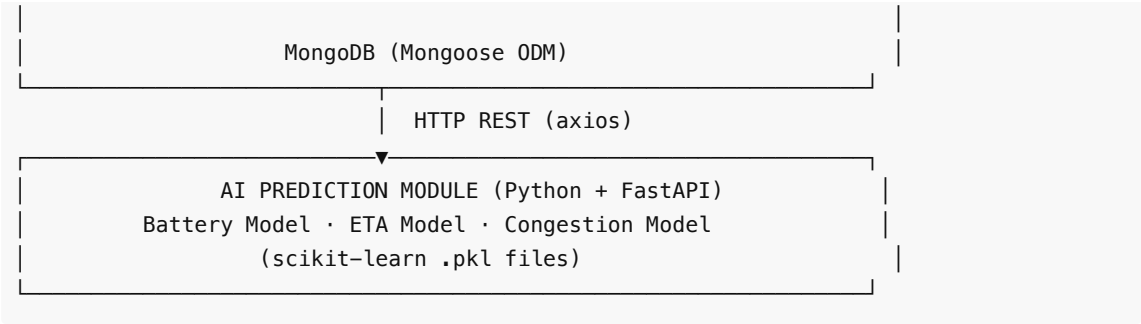
SkyTrace is a real-time, AI-powered Air Traffic Control (ATC) system for autonomous drone deliveries across the IIT Kanpur campus. It handles route planning, collision avoidance, battery management, congestion prediction, and live fleet monitoring — all through a unified dashboard.

Table of Contents

- 1. [System Architecture](#)
- 2. [Tech Stack](#)
- 3. [Feature Deep-Dive](#)
 - [A* Pathfinding & NFZ Avoidance](#)
 - [3D Drone Simulation Engine](#)
 - [AI Prediction Module](#)
 - [Altitude Lane Management](#)
 - [Real-Time Fleet Map](#)
 - [Mission & Order Pipeline](#)
 - [Battery Failsafe & Power Station](#)
 - [3D Collision Detection](#)
 - [Evaluation Scenario Engine](#)
 - [Safety Zone Management](#)
- 4. [Campus Map Data](#)
- 5. [API Reference](#)
- 6. [Running the Project](#)
- 7. [Project Structure](#)

System Architecture





Tech Stack

Layer	Technology
Frontend	React 18, Vite, Leaflet.js, React-Leaflet, Framer Motion, Lucide Icons, Axios, Socket.io-client
Backend	Node.js, Express 5, Socket.io 4, Mongoose 9, Winston (logging), Zod (validation), JWT, bcryptjs
Database	MongoDB Atlas (cloud) / local MongoDB
AI Module	Python 3, FastAPI, scikit-learn, joblib, NumPy, pandas, uvicorn
Map Data	IIT Kanpur campus extracted from OSM via Drone_Map_IITK final.ipynb (Jupyter + folium)
Dev Tools	nodemon, ESMmodules (type: "module")

Feature Deep-Dive

1. A* Pathfinding & NFZ Avoidance

File: backend/services/navigation.service.js

How it works:

A 150×150 grid is overlaid on the IIT Kanpur campus (bounds from `mapConfig.js`). Each cell is classified as either **open (0)** or **blocked (1)**.

Grid Size: 150 × 150 cells
Campus Bounds: lat [26.506, 26.522] × lng [80.222, 80.252]

Grid building process:

- All 9 NFZ polygon zones from `safety.config.js` are rasterized onto the grid using a point-in-polygon check
- A **safety buffer** of 2 extra cells is expanded around each NFZ to prevent drones flying too close to restricted boundaries
- Start and end cells are explicitly forced open — preventing pathfinding failures when hubs are near NFZ edges

A Search:*

- 8-directional movement (diagonals allowed, cost = 1.414)
- Heuristic: Euclidean distance to goal
- Open set sorted by $f = g + h$ each iteration

Path Smoothing (Turning-Point Preserval): Rather than keeping every Nth point, the algorithm:

- Detects **direction changes** in the raw path (turning points = NFZ boundary curves)
- Keeps all turning points + every $\text{floor}(n/50)$ -th point for uniform spacing
- Result: up to 50 semantically meaningful waypoints that trace the exact detour shape

Why no fallbacks?

This is the only routing algorithm. If A* cannot find a route, the mission is rejected with an error. No graph BFS, no straight-line shortcut — this guarantees drones can never silently cut through an NFZ.

2. 3D Drone Simulation Engine

File: backend/services/drone3DService.js

The simulation engine moves drones along their computed A* waypoints in real time and manages all mid-flight state.

Movement model:

Speed: 10 m/s
Tick rate: 1 second
per tick: position advances by $(\text{speed} \times 1\text{s})$ along the 3D path

Each tick:

1. Current position interpolated between waypoints using Haversine geometry
2. `drone_position_3d` event emitted to all frontend clients via Socket.io
3. Battery drained per metre flown (ML model or fallback 0.02%/m)
4. Altitude compared against other active drones → collision detection triggered

Socket.io Events:

Event	Trigger
<code>drone_position_3d</code>	Every 1-second tick
<code>altitude_change</code>	When drone's lane is reassigned mid-flight
<code>drone_low_battery</code>	When battery drops below 15%
<code>drone_charging</code>	Every tick during Power Station charge
<code>drone_charging_done</code>	When battery reaches 100% after charge

3. AI Prediction Module

File: ai-prediction/api.py (FastAPI server, port 8000)

Three pre-trained scikit-learn models serve predictions via REST API:

Battery Model (`battery_model.pkl`)

- **Input:** distance (km), weight (kg), drone speed, altitude
- **Output:** `batteryUsed (%)`, `batteryAfter (%)`
- **Used by:** `mission.service.js` before dispatch — checks if drone has enough charge

ETA Model (`eta_model.pkl`)

- **Input:** distance, congestion score, altitude, time-of-day
- **Output:** estimated arrival time (ISO timestamp)
- **Used by:** Mission creation to show delivery ETA in UI

Congestion Model (`congestion_model.pkl`)

- **Input:** hub ID, time slot, current drone density near hub
- **Output:** congestion probability score (0–1)
- **Used by:** Lane assignment and route selection to prefer low-congestion corridors

Training data: `drone_data.csv` — 10,000+ synthetic drone delivery records generated by `synthetic_data.py`

4. Altitude Lane Management

File: `backend/services/altitudeManager.js`

Config: `backend/config/safety.config.js`

Campus airspace is divided into **time-slotted altitude lanes** to prevent horizontal and vertical drone conflicts:

```
ALTITUDE_LANES = [  
  { id: 1, altitude: 20m, maxDrones: 3 },  
  { id: 2, altitude: 30m, maxDrones: 3 },  
  { id: 3, altitude: 40m, maxDrones: 3 },  
  { id: 4, altitude: 50m, maxDrones: 3 },  
  { id: 5, altitude: 60m, maxDrones: 3 },  
]  
TIME_SLOT_DURATION = 30 seconds
```

Lane assignment algorithm:

1. Calculate current time slot index → `Math.floor(Date.now() / 30000)`
 2. Check occupancy table for each lane in the requested slot
 3. Assign the first lane with available capacity
 4. Reserve the slot with the drone's ID
 5. Emit `altitude_change` if a different altitude than the drone's preferred one is assigned
-

5. Real-Time Fleet Map

File: `frontend/src/components/LiveFleetMap.jsx`

Built using **Leaflet.js + React-Leaflet** with a custom dark-themed tile layer.

What's displayed:

- **64 campus nodes** as interactive markers (hubs highlighted in blue)
- **9 NFZ polygons** rendered as red filled zones with dashed borders

- **Live drone markers** that update position every second from Socket.io
- **Flight path polylines** showing A* route for each active drone
- **Congestion overlay** — grid cells color-coded by occupancy density
- **Altitude legend** showing active lane distribution

Real-time data flow:

```
drone3DService (backend)
  → Socket.io emit("drone_position_3d")
  → LiveFleetMap listener
    → React state update
      → Marker re-render on map
```

6. Mission & Order Pipeline

Files: backend/services/mission.service.js , mission.controller.js

Full lifecycle from customer order to delivery completion:

```
POST /api/v1/orders
  ↓
  Order created in MongoDB (status: "pending")
  ↓
  MissionService.createMission(orderId)
  ↓
  Find idle drone with sufficient battery
  ↓
  Call AI API → battery prediction + ETA
  ↓
  Call NavigationService.get3DRoute() → A* path
  ↓
  Assign altitude lane + time slot
  ↓
  Mission record created in MongoDB
  ↓
  drone3DService.startDrone3D(path) → simulation begins
  ↓
  Socket.io events → frontend live updates
```

Pre-flight battery check:

The AI-predicted `batteryUsed` is compared against `drone.batteryLevel` . If insufficient (< 10%), the mission is rejected. If between 10–threshold, the drone launches with a warning — the 3D service will divert it to the Power Station mid-flight.

7. Battery Failsafe & Power Station

File: backend/services/drone3DService.js

When battery drops below **15%** during flight:

1. Emit "drone_low_battery" → frontend shows RED battery badge
2. Pause current mission waypoints

3. Compute new A* path → current position → Power Station (lat: 26.5090, lng: 80.2375)
4. Fly drone to Power Station
5. Begin charging: 100% / 3600s = 0.0278% per second
6. Emit "drone_charging" every tick with charge progress
7. When battery = 100% → emit "drone_charging_done"
8. Resume original mission from Power Station → original destination

3-hop journey: Source → ⚡ Power Station → Destination

8. 3D Collision Detection

File: backend/services/collision3D.js

Every simulation tick, each drone's 3D position is compared against all other active drones:

Separation threshold: 50m horizontal + 10m vertical

If two drones come within the threshold:

1. safety_alert event emitted with both drone IDs
2. The lower-priority drone is bumped to the next altitude lane
3. altitude_change event emitted → badge on map changes

9. Evaluation Scenario Engine

Files: backend/services/scenario.service.js , scenario.controller.js , scenario.routes.js

Frontend: frontend/src/components/ScenarioPanel.jsx

One-click scenarios for live demos. Each scenario:

1. Clears all existing Missions and Orders from MongoDB
2. Resets all drones to idle + 100% battery
3. Launches the scenario sequence

Scenario	Route	POST /api/v1/scenarios/run/
● High Traffic Density	8 inter-hub deliveries (all 5 hubs) every 2.5s	traffic
● Altitude Traffic Conflict	8 drones on crossing routes sharing altitudes (600ms gaps)	altitude
● Hub Rush Hour	6 campus locations (H1, LHC, H5, H9, GH, GATE) → Hub Central simultaneously	congestion
● Critical Battery Failsafe	Guest House → Hall 9 (~1800m), 35% battery, 5kg payload	battery

Reset endpoint: POST /api/v1/scenarios/reset — clears environment without launching a scenario.

All locations use real CAMPUS_NODES from campusGraph.config.js .

10. Safety Zone Management

File: backend/config/safety.config.js , frontend/src/components/SafetyZones.jsx

9 No-Fly Zones extracted from Drone_Map_IITK final.ipynb :

- NFZ-Admin Block
- NFZ-Library Complex
- NFZ-Lecture Hall Complex (LHC)
- NFZ-Faculty Building
- NFZ-Main Gate Perimeter
- NFZ-OAT Amphitheatre
- NFZ-Southern Residential
- NFZ-Sports Complex
- NFZ-Research Park

Each NFZ is defined as a **12-point polygon** (circle approximation). The A* grid rasterizes these and adds a **2-cell safety buffer** around each, so drones never fly closer than ~15m to any boundary.

Campus Map Data

Extracted from Drone_Map_IITK final.ipynb (Jupyter notebook, 64 nodes):

Category	Count
Campus Nodes	64
Drone Hubs	5 (North, South, East, West, Central)
NFZ Zones	9
OSM Paths	39
Grid Paths	805
Safe Airspace	58.69%

Hub Coordinates:

Hub	Location	Coordinates
HUB-N	Hub North	26.5200, 80.2320
HUB-S	Hub South	26.5088, 80.2330
HUB-E	Hub East	26.5148, 80.2392
HUB-W	Hub West	26.5148, 80.2248
HUB-C	Hub Central	26.5140, 80.2318

API Reference

Orders

Method	Endpoint	Description
--------	----------	-------------

POST	/api/v1/orders	Create new delivery order
GET	/api/v1/orders	Get all orders
GET	/api/v1/orders/:id	Get order by ID

Drones

Method	Endpoint	Description
GET	/api/v1/drones	Get all drones + live status
POST	/api/v1/drones	Register new drone
PUT	/api/v1/drones/:id	Update drone config

Missions

Method	Endpoint	Description
GET	/api/v1/missions	Get all missions
POST	/api/v1/missions	Dispatch mission for an order

Scenarios

Method	Endpoint	Description
POST	/api/v1/scenarios/run/:name	Launch scenario (traffic/altitude/congestion/battery)
POST	/api/v1/scenarios/reset	Reset all drones and clear missions

Safety

Method	Endpoint	Description
GET	/api/v1/safety/nfz	Get all NFZ zone data
GET	/api/v1/safety/alerts	Get recent safety violations

AI Module (FastAPI, port 8000)

Method	Endpoint	Description
POST	/predict/battery	Predict battery usage for a mission
POST	/predict/eta	Predict delivery ETA
POST	/predict/congestion	Predict hub congestion score

Running the Project

Prerequisites

- Node.js ≥ 18
- Python ≥ 3.10
- MongoDB (local or Atlas URI in `.env`)

1. Backend

```
cd backend
npm install
# Create .env with MONGO_URI and PORT
npm run dev          # starts on port 5000
```

2. AI Prediction Module

```
cd ai-prediction
pip install fastapi uvicorn scikit-learn joblib numpy pandas
uvicorn api:app --host 0.0.0.0 --port 8000 --reload
```

3. Frontend

```
cd frontend
npm install
npm run dev          # starts on port 5173
```

4. Seed Drones

```
cd backend
node seedDrones.js  # creates 12 drones in MongoDB
```

5. Open Dashboard

`http://localhost:5173`

Project Structure

```
IIT-Kanpur/
├── backend/
│   ├── config/
│   │   ├── campusGraph.config.js  # 64 real campus nodes + edges
│   │   ├── safety.config.js       # 9 NFZ polygons + altitude lanes
│   │   ├── buildings.config.js    # Building obstacle data
│   │   └── mapConfig.js           # Campus bounds + map center
│   ├── controllers/               # Express route handlers
│   ├── models/                   # Mongoose schemas (Drone, Order, Mission)
│   ├── routes/                   # Express routers
│   ├── services/
│   │   └── navigation.service.js  # A* pathfinding (SOLE algorithm)
```

```

├── drone3DService.js      # 3D simulation + battery management
├── mission.service.js    # Mission dispatch pipeline
├── scenario.service.js   # 4 evaluation scenarios
├── altitudeManager.js    # Lane + time-slot ATC
├── collision3D.js        # 3D proximity detection
├── ai.service.js         # Bridge to Python AI API
├── safety.service.js     # NFZ violation detection
├── utils/
│   ├── distanceCalculator.js # Haversine + path distance
│   └── logger.js          # Winston logger
└── server.js             # Express + Socket.io bootstrap

├── frontend/
│   └── src/
│       ├── components/
│       │   ├── LiveFleetMap.jsx    # Real-time Leaflet map
│       │   ├── ScenarioPanel.jsx   # Evaluation scenario buttons
│       │   ├── FleetManagement.jsx # Drone fleet table
│       │   ├── MissionPlanner.jsx  # Manual mission dispatch form
│       │   ├── AnalyticsPage.jsx   # Stats + charts
│       │   ├── SafetyZones.jsx     # NFZ management view
│       │   ├── SettingsPage.jsx    # System configuration
│       │   └── Sidebar.jsx         # Grouped navigation
│       └── config/
│           └── mapConfig.js        # API_URL + map bounds (frontend)
└── ai-prediction/
    ├── api.py                 # FastAPI server
    ├── battery_model.pkl      # Trained battery drain model
    ├── eta_model.pkl          # Trained ETA model
    ├── congestion_model.pkl   # Trained congestion model
    ├── synthetic_data.py      # Training data generator
    └── drone_data.csv         # 10,000+ training records

└── Drone_Map_IITK final.ipynb # Campus map extraction notebook

```

Team

Member	Responsibility
Gaurav	Backend architecture, A* pathfinding, scenario engine, navigation service
Kratika	AI prediction module, ML model training, FastAPI server
Abhishek	Frontend dashboard, real-time map, Socket.io integration
Mahee	Database design, fleet management, mission pipeline